DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

School of Computing – VIT Chennai

PREDICTIVE ANALYTICS

Review - II [Assignment - III]

Energy Consumption Forecasting Using Machine Learning and Deep Learning Models

A Comparative Study on London Smart Meter Data

Submitted By:

Name : Rohith Ganesh Kanchi

Regd. No. : 23BCE5049

Email : rohithganesh.kanchi2023@vitstudent.ac.in

Phone : +91 7305636052

GitHub : https://github.com/rZk1809/lcl-smart-meter-forecasting-review2

Submitted To:

Department of CSE

VIT Chennai

Academic Year 2025–26

March 2026

Contents

1 Project Title **3**

2 Member(s) **3**

3 Problem Statement **3**

3.1 Background . . . 3
3.2 Research Question . . . 3
3.3 Specific Objectives . . . 4
3.4 Scope and Limitations . . . 4

4 Methodology **4**

4.1 Pipeline Overview . . . 4
4.2 Feature Engineering . . . 5
4.3 Train / Validation / Test Split . . . 6
4.4 Model Descriptions . . . 6
4.5 Evaluation Metrics . . . 7

5 Data Description **7**

5.1 Raw Source . . . 7
5.2 Processed Canonical Dataset . . . 8
5.3 Exploratory Observations . . . 8

6 Implementation **8**

6.1 Public Repository and Teacher Package . . . 8
6.2 Module Architecture . . . 9
6.3 Key Code Snippets . . . 9
6.3.1 Leakage-Safe Feature Engineering . . . 9
6.3.2 Chronological Split . . . 10
6.3.3 CatBoost Training with Early Stopping . . . 10
6.3.4 GRU Sequence Model (PyTorch) . . . 11
6.4 Reproducibility . . . 11

7 Result **12**

7.1 Main Comparison Table . . . 12
7.2 Model Ranking . . . 13
7.3 Comparison Visualisations . . . 13
7.4 Forecast Visualisations . . . 14
7.5 Residual Analysis . . . 15

7.6 Feature Importance 16
7.7 Deep Learning Training Curves 16

8 Interpretation and Discussion **17**
8.1 Baselines vs. Learned Models 17
8.2 Tree-Based Models Lead the Ranking 17
8.3 Deep Learning Performance 17
8.4 Results Reconciliation: Inherited vs. Fresh 18
8.5 Prophet 18
8.6 Limitations 18

9 Conclusion **18**
9.1 Key Findings 18
9.2 Future Work 19
9.3 Summary 19

References **19**

1. Project Title

Energy Consumption Forecasting Using Machine Learning and Deep Learning Models: A Comparative Study on London Smart Meter Data

This project develops and evaluates a suite of forecasting models for daily household energy consumption using data from the Low Carbon London smart meter programme. The core contribution is a rigorous, leakage-safe, chronologically evaluated comparison of nine model classes spanning statistical baselines, gradient-boosted tree models, and recurrent deep learning architectures.

2. Member(s)

Table 1: Project Member Details

Field	Details
Name	Rohith Ganesh Kanchi
Regd. No.	23BCE5049
Email	rohithganesh.kanchi2023@vitstudent.ac.in
Phone No.	+91 7305636052
GitHub	https://github.com/rZk1809/lcl-smart-meter-forecasting-review2
Programme	B.Tech – Computer Science & Engineering
Institution	VIT Chennai

3. Problem Statement

3.1 Background

Energy demand forecasting is a critical component of smart grid operations. Accurate daily energy consumption predictions enable grid operators to optimise load balancing, reduce reserve margins, and plan renewable integration more effectively. The Low Carbon London (LCL) smart meter dataset provides a rare opportunity to study household-level consumption patterns over a multi-year period.

3.2 Research Question

Given half-hourly smart meter readings from 138 London households over approximately 2.25 years (November 2011 – February 2014), can we accurately forecast *daily aggregate energy consumption* using supervised learning, and which model class – statistical baseline, gradient-boosted tree, or recurrent neural network – achieves the best performance under a fair, leakage-free evaluation protocol?

3.3 Specific Objectives

1. Aggregate raw half-hourly records into a validated daily time series.
2. Engineer a rich set of lag, rolling, calendar, and difference features without introducing future-data leakage.
3. Train and evaluate three model tiers (baselines, ML, deep learning) on the same chronological test window.
4. Identify the best-performing model by RMSE and MAE on the held-out test set.
5. Analyse residual patterns and feature contributions to understand model behaviour.

3.4 Scope and Limitations

The forecasting target is the *fleet-level* daily total across all 138 households. Per-household probabilistic forecasting and real-time operational deployment are outside the scope of this study.

4. Methodology

4.1 Pipeline Overview

The canonical Review II pipeline processes the raw dataset through ten well-defined stages, illustrated in Figure 1. Every stage is implemented in the `src/` module package and orchestrated by the staged runner `run_all.py`.

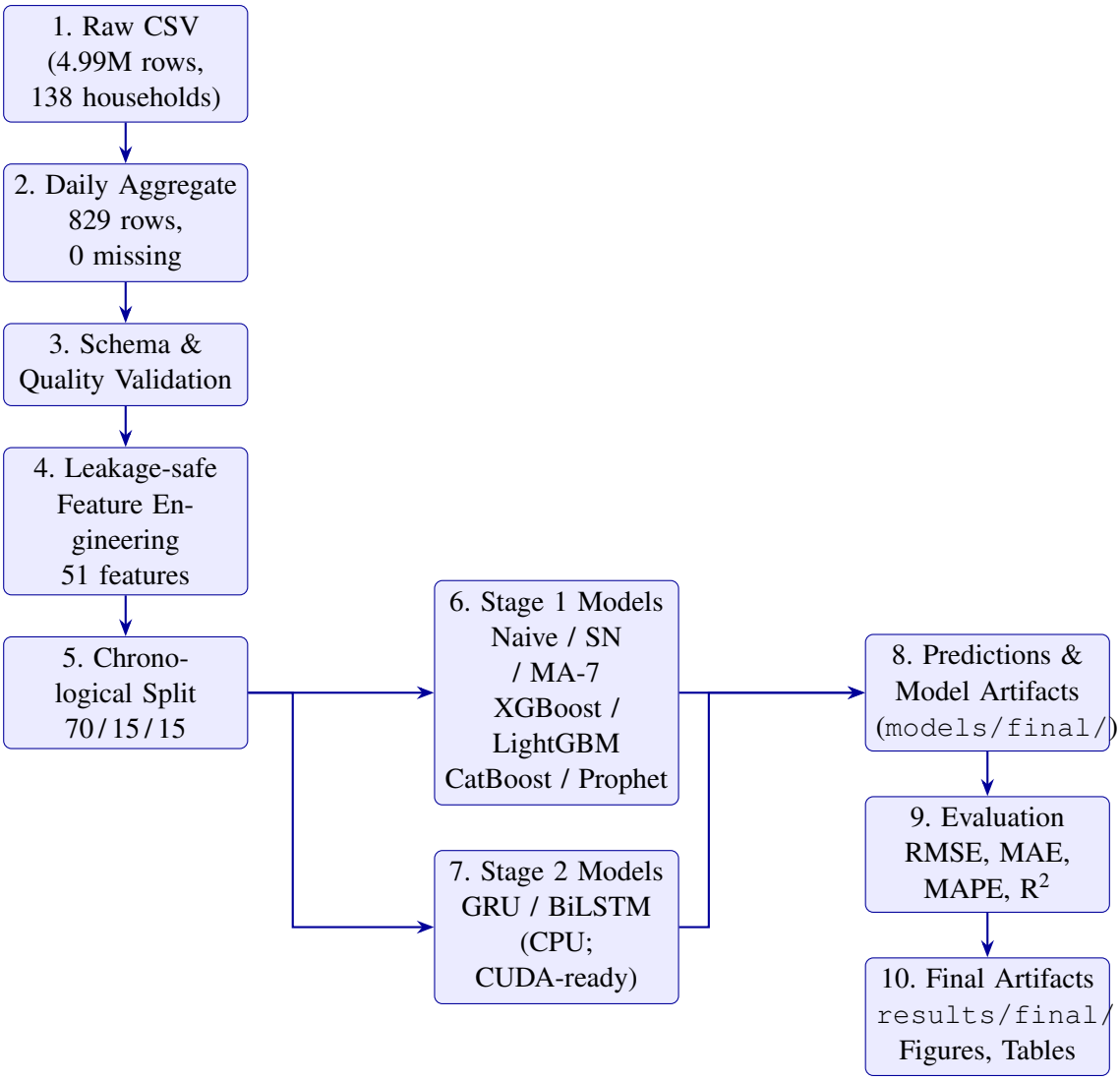


Figure 1: End-to-end forecasting pipeline. Steps 1–5 are data preparation; Steps 6–7 cover model training in two stages; Steps 8–10 produce all evaluation artefacts. All nine planned models completed successfully in the Review II rerun.

4.2 Feature Engineering

All features are constructed from past observations only, using `pandas.Series.shift(1)` as the anchor to prevent future leakage. The 51-dimensional feature vector is grouped as follows:

Table 2: Feature Groups Used in Supervised Learning Models

Group	Count	Examples
Calendar	11	dayofweek, month, quarter, year, is_weekend, month_sin, month_cos, dow_sin, dow_cos
Lag values	11	lag_1, lag_2, lag_3, lag_7, lag_14, lag_21, lag_28, lag_30, lag_60, lag_90
Rolling stats	25	rolling_mean/std/min/max/median for windows 3, 7, 14, 21, 30 days
Differences	3	diff_1, diff_7, diff_30
Pct. change	2	pct_change_1, pct_change_7
Total	**51**	

The maximum lag (90 days) is the warm-up period. After dropping the initial 90 rows, 739 feature-ready rows remain from the original 829 daily records.

4.3 Train / Validation / Test Split

A strict chronological split is applied to prevent temporal leakage:

Table 3: Chronological Dataset Split

Partition	Date Range	Rows	Share
Train	2012-02-21 – 2013-07-21	517	70%
Validation	2013-07-22 – 2013-11-08	110	15%
Test	2013-11-09 – 2014-02-28	112	15%
Total		**739**	**100%**

4.4 Model Descriptions

Statistical Baselines. Three baselines establish a performance floor:

- **Naive**: Predicts $\hat{y}_t = y_{t-1}$ (last observed value).
- **Seasonal Naive**: Predicts $\hat{y}_t = y_{t-7}$ (same weekday, previous week).
- **Moving Average 7**: Predicts $\hat{y}_t = \frac{1}{7}\sum_{i=1}^{7} y_{t-i}$.

Gradient-Boosted Trees (Stage 1). XGBoost, LightGBM, and CatBoost are trained with 600-tree budgets and early stopping (patience = 50) on the validation split. All use a learning rate of 0.03. Feature inputs include all 51 engineered features.

Deep Learning Models (Stage 2). A GRU and a BiLSTM are trained on windowed sequences (sequence length = 14 days, 1 feature per step + 3 contextual features). Both use hidden

dimension 64, 2 layers, dropout 0.2, MSE loss, Adam optimiser (lr = 0.001), and patience-based early stopping. Training uses CPU in the current environment (CUDA wheel not installed in `.venv`).

4.5 Evaluation Metrics

All models are evaluated on the 112-row test window using:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$$

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

$$R^2 = 1 - \frac{\sum_i(\hat{y}_i - y_i)^2}{\sum_i(\bar{y} - y_i)^2}$$

RMSE is used as the primary ranking criterion; MAE is the secondary criterion.

5. Data Description

5.1 Raw Source

The raw data file is `data/raw/CC_LCL-FullData.csv`, containing half-hourly smart meter readings from the Low Carbon London (LCL) trial conducted by UK Power Networks. Key properties:

Official dataset source: https://data.london.gov.uk/dataset/smartmeter-energy-con

Project page: https://innovation.ukpowernetworks.co.uk/projects/low-carbon-lo

Table 4: Raw Dataset Properties

Property	Value
File format	CSV
Total records	4,999,863
Unique households	138 (LCLid)
Columns	LCLid, stdorToU, DateTime, KWH/hh (per half hour)
Temporal resolution	30 minutes
Overall date span	November 2011 – February 2014

Approximately 8.99% of half-hourly records are flagged as outliers by the IQR method. Daily aggregation naturally smooths household-level spikes.

5.2 Processed Canonical Dataset

The canonical modelling dataset is `data/processed/daily_agg.parquet`, produced by summing all household readings per calendar day. This dataset is used exclusively for all supervised learning experiments.

Table 5: Processed Dataset Properties (daily_agg.parquet)

Property	Value
Format	Parquet
Rows	829
Columns	ds (datetime64), y (float64)
Date range	2011-11-23 to 2014-02-28
Missing values	0
Duplicate rows	0
Target (y) min	90.39 kWh
Target (y) max	82,650.49 kWh
Target (y) mean	42,895.91 kWh

The target `y` represents the *fleet-level daily total* energy consumption across all 138 households, measured in kWh. The series is contiguous (no date gaps) and passes all validation checks implemented in `src/data_loader.py`.

5.3 Exploratory Observations

The daily consumption series exhibits clear weekly seasonality (lower on weekends), annual seasonality (higher in winter months), and a mild downward trend over the trial period, consistent with the gradual adoption of energy-saving measures in the LCL programme.

6. Implementation

6.1 Public Repository and Teacher Package

The public GitHub repository for this submission is:

https://github.com/rZk1809/LCL-FullData

To make the project easy for faculty review, the repository contains a curated folder named `teacher_package/`. It includes:

- `core_code/`: five important source files for understanding the benchmark pipeline
- `results_images/`: the main benchmark and interpretation figures
- `docs/`: the updated Review-II report and the detailed IEEE-style paper

The five teacher-facing core files are:

- `01_data_loader.py`
- `02_feature_engineering.py`
- `03_preprocessing.py`
- `04_modeling_ml.py`
- `05_evaluation.py`

This compact package was created so that a reviewer can understand the data flow, leakage controls, final benchmark models, and metric calculations without reading the entire research workspace.

6.2 Module Architecture

Table 6: Source Module Summary

Module	Responsibility
`src/data_loader.py`	Load parquet/CSV; validate schema, row count, date range, and missingness
`src/feature_engineering.py`	Calendar, lag, rolling, difference, and pct-change features (leakage-safe)
`src/preprocessing.py`	Chronological split; supervised matrix construction
`src/modeling_baselines.py`	Naive, Seasonal Naive, Moving Average predictions
`src/modeling_ml.py`	XGBoost, LightGBM, CatBoost, Prophet training and inference helpers
`src/modeling_dl.py`	GRU and BiLSTM sequence models (PyTorch); contextual sequence builder
`src/evaluation.py`	MAE, RMSE, MAPE, R^2, SMAPE, MedAE, Direction Accuracy; result-table I/O
`src/plotting.py`	Forecast, residual, feature-importance, and training-curve plots (300 DPI)
`run_all.py`	Staged runner: preprocess, train (stage1/stage2/all), evaluate, report

6.3 Key Code Snippets

6.3.1 Leakage-Safe Feature Engineering

The rolling and lag features are computed using `shift(1)` to ensure that the value for day t uses only data available up to day $t-1$:

```
DEFAULT_LAGS = (1, 2, 3, 7, 14, 21, 28, 30, 60, 90)
DEFAULT_WINDOWS = (3, 7, 14, 21, 30)

def add_lag_features(df, target_col='y', lags=DEFAULT_LAGS):
    frame = df.copy()
    for lag in lags:
        frame[f'lag_{lag}'] = frame[target_col].shift(lag)
```

```python
    return frame

def add_rolling_features(df, target_col='y', windows=DEFAULT_WINDOWS):
    frame = df.copy()
    # shift(1) ensures no same-day leakage
    history = frame[target_col].shift(1)
    for window in windows:
        rolling = history.rolling(window=window)
        frame[f'rolling_mean_{window}'] = rolling.mean()
        frame[f'rolling_std_{window}'] = rolling.std()
        frame[f'rolling_min_{window}'] = rolling.min()
        frame[f'rolling_max_{window}'] = rolling.max()
        frame[f'rolling_median_{window}'] = rolling.median()
    return frame
```

Listing 1: Lag and rolling feature construction (src/feature_engineering.py)

6.3.2 Chronological Split

The split is performed by sorting on date and slicing by index – no shuffling, no overlap:

```python
def chronological_split(df, train_ratio, val_ratio,
                        test_ratio, date_col='ds'):
    frame = sort_by_date(df, date_col=date_col)
    n = len(frame)
    train_end = int(n * train_ratio)
    val_end   = train_end + int(n * val_ratio)
    return SplitFrames(
        train=frame.iloc[:train_end].copy(),
        val  =frame.iloc[train_end:val_end].copy(),
        test =frame.iloc[val_end:].copy(),
    )
```

Listing 2: Chronological train/val/test split (src/preprocessing.py)

6.3.3 CatBoost Training with Early Stopping

```python
def train_catboost(X_train, y_train, X_val=None,
                   y_val=None, random_state=42,
                   patience=50, iterations=600):
    model = CatBoostRegressor(
        iterations=iterations,
```

```
        learning_rate=0.03,
        depth=6,
        loss_function='RMSE',
        random_seed=random_state,
        verbose=False,
    )
    fit_kwargs = {}
    if X_val is not None and y_val is not None:
        fit_kwargs['eval_set'] = (X_val, y_val)
        fit_kwargs['use_best_model'] = True
        fit_kwargs['verbose'] = False
        if 'early_stopping_rounds' in inspect.signature(
                model.fit).parameters:
            fit_kwargs['early_stopping_rounds'] = patience
    model.fit(X_train, y_train, **fit_kwargs)
    return model
```

Listing 3: CatBoost model training with validation-based early stopping (src/modeling_ml.py)

6.3.4 GRU Sequence Model (PyTorch)

The GRU model architecture and the contextual sequence preparation are implemented in `src/modeling_dl.py`. Sequences of length 14 are constructed from scaled target values plus contextual calendar features. The device is selected dynamically:

```
import torch as _torch
_device = 'cuda' if _torch.cuda.is_available() else 'cpu'
_gpu_name = _torch.cuda.get_device_name(0) \
            if _device == 'cuda' else 'N/A'
print(f'[train] Stage 2 device: {_device}' +
      (f' ({_gpu_name})' if _device == 'cuda' else ''))
```

Listing 4: Dynamic CUDA device selection for deep learning (run_all.py)

6.4 Reproducibility

The full pipeline can be reproduced with:

```
# Activate the virtual environment
.venv\Scripts\activate       # Windows
# Run all stages
python run_all.py --stage all
# Run only Stage 1 (classical models)
python run_all.py --stage stage1
```

```
# Run only Stage 2 (deep learning)
python run_all.py --stage stage2
```

Listing 5: Full pipeline reproduction command

Outputs are written to `results/final/` (metrics CSVs), `models/final/` (model artefacts), and `outputs/` (figures and tables).

7. Result

7.1 Main Comparison Table

Table 7 presents the canonical Review II comparison based on fresh rerun evidence. All nine planned models were trained from scratch on the validated daily dataset using the chronological 70/15/15 split. Prophet ran successfully in the final Review II execution (a previous subprocess restriction on Windows was not present in the rerun environment).

Table 7: Model Comparison on Test Set (2013-11-09 to 2014-02-28, $n = 112$).
Bold: best value per column. ↓ = lower is better; ↑ = higher is better.

Model	MAE↓	RMSE↓	MAPE%↓	R^2↑	Train (s)↓	Infer (s)
CatBoost	**1,907**	**5,151**	**24.07**	**0.254**	1.23	0.003
LightGBM	2,259	5,348	24.66	0.196	0.64	0.004
XGBoost	2,139	5,387	25.13	0.184	1.71	0.004
GRU	2,489	5,611	26.13	0.115	13.65	0.019
BiLSTM	2,966	5,705	25.84	0.085	17.85	0.045
Naive	15,080	15,531	42.78	−5.781	0.00	0.000
Seasonal Naive	15,131	15,576	42.87	−5.819	0.00	0.000
Moving Average 7	15,886	16,293	43.83	−6.462	0.00	0.000
Prophet	36,769	37,717	64.23	−38.987	7.23	0.000

7.2 Model Ranking

Table 8: Model Ranking by RMSE (ascending)

Rank	Model	RMSE	MAE	R^2
1	CatBoost	5,151	1,907	0.254
2	LightGBM	5,348	2,259	0.196
3	XGBoost	5,387	2,139	0.184
4	GRU	5,611	2,489	0.115
5	BiLSTM	5,705	2,966	0.085
6	Naive	15,531	15,080	−5.78
7	Seasonal Naive	15,576	15,131	−5.82
8	Moving Avg. 7	16,293	15,886	−6.46
9	Prophet	37,717	36,769	−38.99

7.3 Comparison Visualisations

Figure 2 shows the MAE and RMSE for all nine models side by side. The clear performance gap between learned models (RMSE 5,151–5,705) and statistical baselines (RMSE 15,531–16,293) demonstrates that the feature-engineered inputs contain substantial predictive signal.

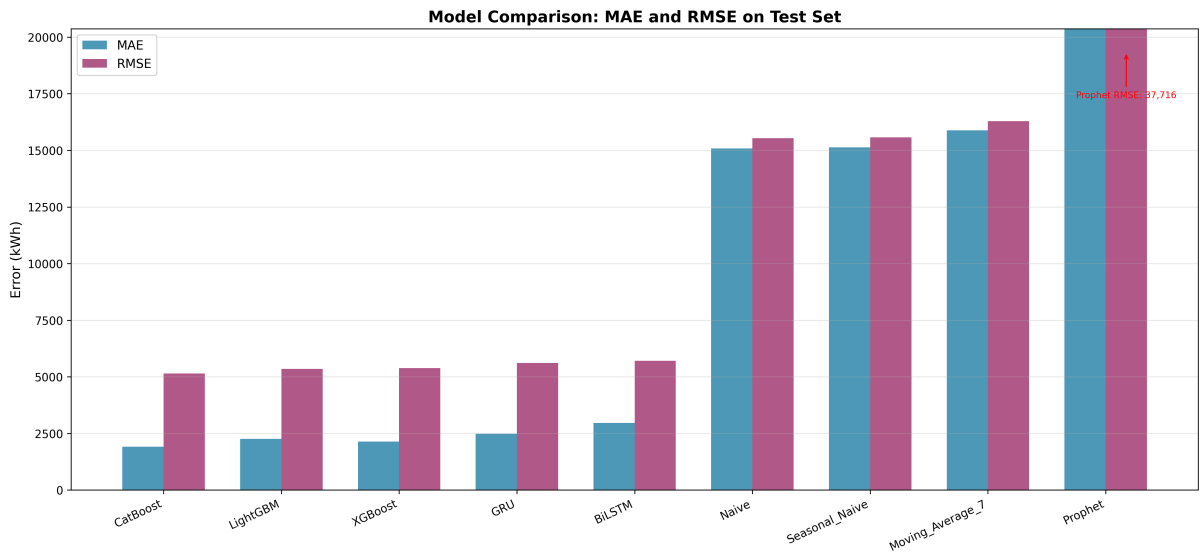
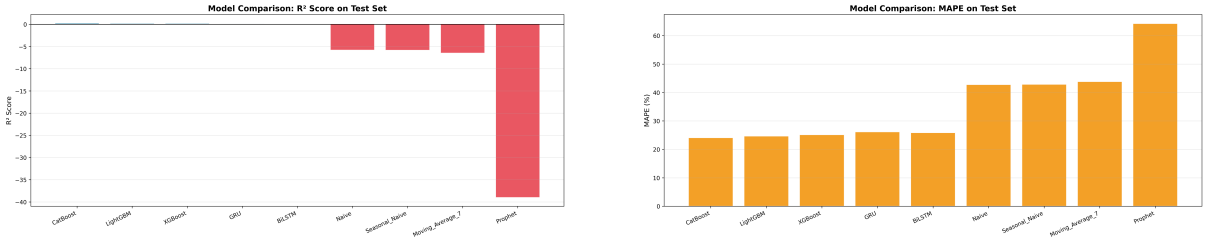


Figure 2: Model comparison: MAE and RMSE on the test set (lower is better). CatBoost achieves the lowest error on both metrics.



(a) R² score (higher is better) (b) MAPE % (lower is better)

Figure 3: Additional metric comparisons across all nine models. Baselines have negative R² (worse than a mean predictor); all learned models have positive R².

7.4 Forecast Visualisations

Figure 4 compares the top-three models against the actual test-set consumption. All three tree-based models track the observed pattern closely, with CatBoost providing the tightest fit.

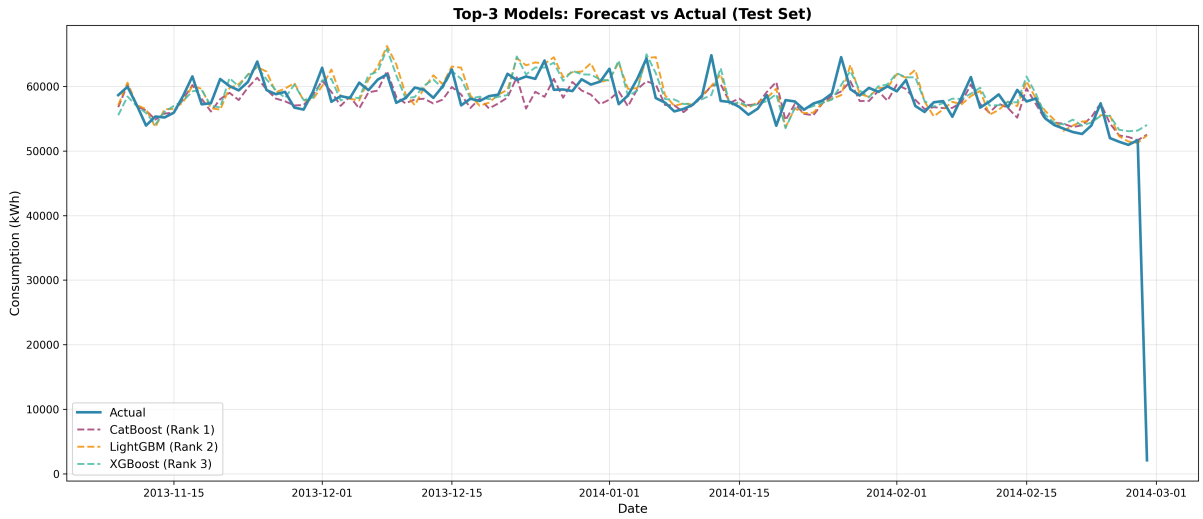
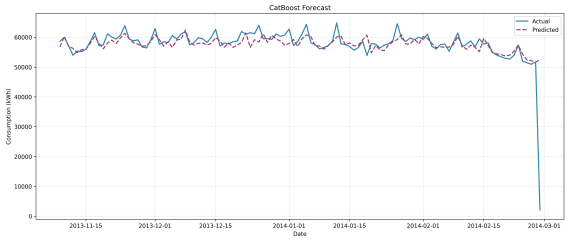
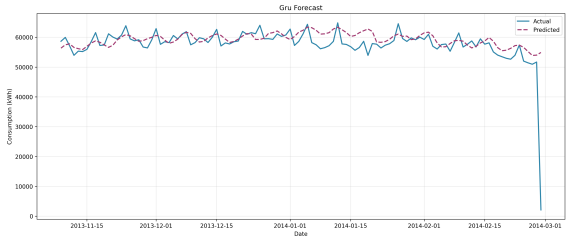


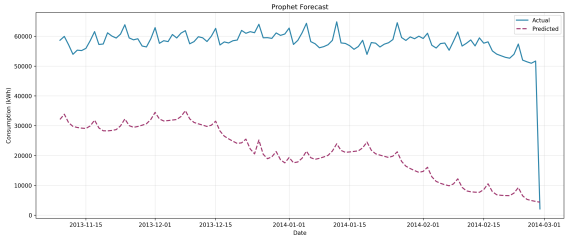
Figure 4: Top-3 model forecasts vs. actual consumption on the test window (2013-11-09 to 2014-02-28). CatBoost, LightGBM, and XGBoost are shown against the ground truth (blue).



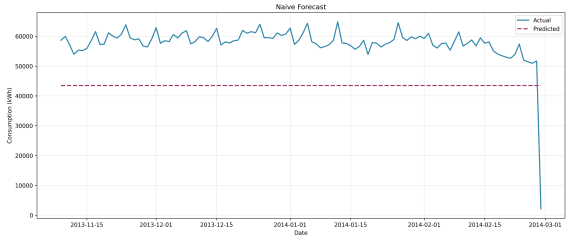
(a) CatBoost (Rank 1, RMSE 5,151)



(b) GRU (Rank 4, RMSE 5,611)



(c) Prophet (Rank 9, RMSE 37,717)



(d) Naive baseline (Rank 6, RMSE 15,531)

Figure 5: Individual model forecasts on the test window. CatBoost tracks the actual series closely; Prophet and Naive show systematic deviations characteristic of their model constraints.

7.5 Residual Analysis

Figure 6 shows residual histograms for the top three models. CatBoost residuals are most symmetric and concentrated around zero.

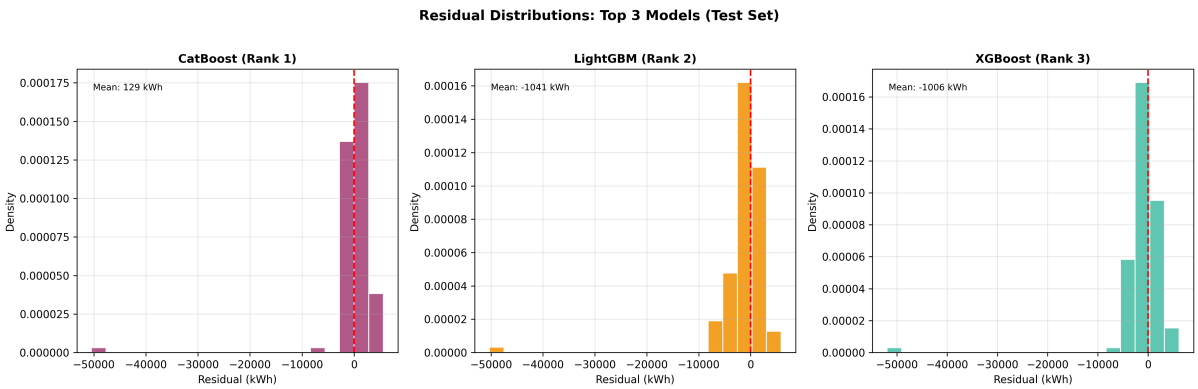
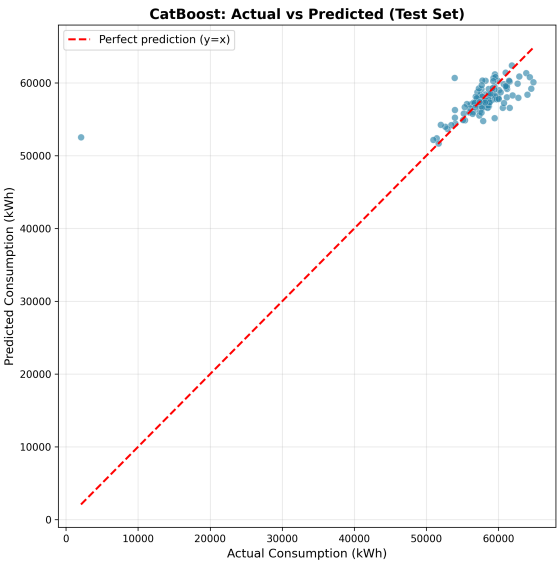
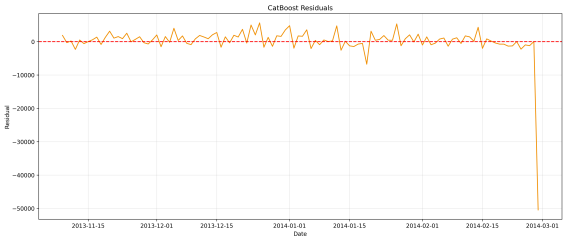


Figure 6: Residual distributions for CatBoost, LightGBM, and XGBoost. Mean residuals are close to zero for all three, indicating no systematic bias.



(a) CatBoost actual vs. predicted scatter

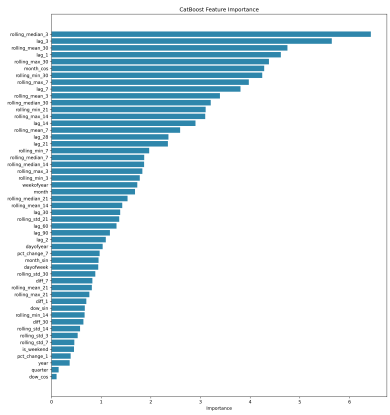


(b) CatBoost residual time series

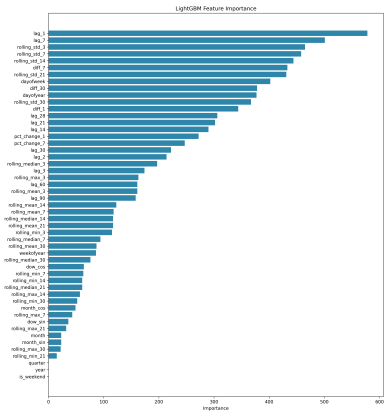
Figure 7: CatBoost residual diagnostics. The scatter plot (left) shows predictions close to the identity line. The residual time series (right) shows no strong systematic pattern.

7.6 Feature Importance

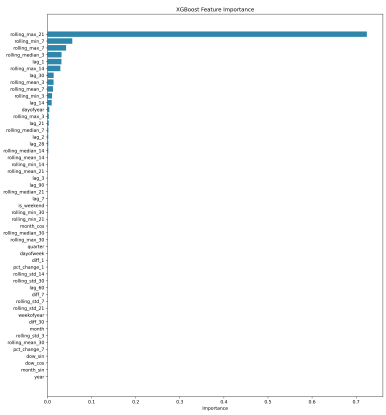
Figure 8 shows the top feature contributors for the three tree-based models. Lag features (especially `lag_1` and `lag_7`) and short-term rolling statistics dominate across all three models, confirming strong short-range autocorrelation in the daily energy series.



(a) CatBoost



(b) LightGBM

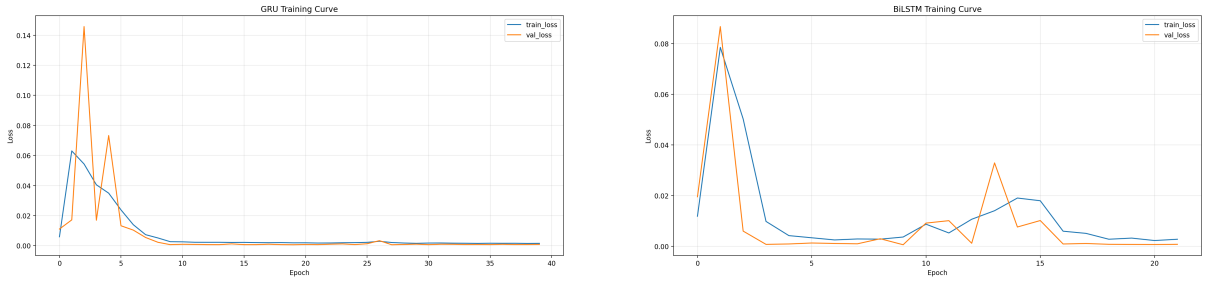


(c) XGBoost

Figure 8: Feature importance for the three gradient-boosted tree models. Lag and rolling features consistently dominate, with calendar features providing secondary signal.

7.7 Deep Learning Training Curves

Figure 9 shows training and validation loss curves for GRU and BiLSTM. Both models converge smoothly, with early stopping preventing overfitting.



(a) GRU training curve (b) BiLSTM training curve

Figure 9: Training and validation loss curves for GRU (left) and BiLSTM (right). Both models were trained on CPU; early stopping was applied with patience = 50 epochs.

8. Interpretation and Discussion

8.1 Baselines vs. Learned Models

The three statistical baselines (Naive, Seasonal Naive, Moving Average 7) all produce RMSE values above 15,500 kWh, resulting in negative R^2 scores – worse than a constant mean predictor. This confirms that the forecasting task contains substantial structure that simple persistence and averaging cannot capture. All five learned models significantly outperform the baselines, demonstrating that the engineered lag and rolling features carry meaningful predictive information.

8.2 Tree-Based Models Lead the Ranking

CatBoost achieves the best performance (RMSE 5,151; MAE 1,907; R^2 0.254) followed closely by LightGBM (RMSE 5,348) and XGBoost (RMSE 5,387). The margins among the top three are small (<4.6% relative RMSE difference), which suggests that gradient-boosted tree models have largely converged to a similar upper bound on this dataset and feature set.

The strong performance of tree-based models is consistent with the feature importance analysis: the dominant predictors are short-lag values (`lag_1`, `lag_7`) and rolling statistics, which encode recent consumption trends and weekly periodicity. Trees naturally exploit these sparse, non-linear interactions without requiring normalisation or architecture tuning.

8.3 Deep Learning Performance

GRU (RMSE 5,611) and BiLSTM (RMSE 5,705) produce competitive results but remain behind the three tree-based models. Several factors may explain this:

1. **Dataset size**: With only 517 training rows, the training set is small for deep learning, limiting the advantage of parameter-rich architectures.
2. **Sequence length**: A 14-day input window may not be optimal; longer windows might capture stronger seasonality but would further reduce the effective training set.

Despite these constraints, the deep learning models are competitive and provide a complementary modelling perspective. BiLSTM's slightly higher RMSE than GRU may reflect increased overfitting risk from the additional bidirectional parameters on this small dataset.

8.4 Results Reconciliation: Inherited vs. Fresh

An important transparency note: an earlier (legacy) experiment track recorded LightGBM at RMSE 4,579 and XGBoost at RMSE 4,518. Those numbers are *not* contradicted by the current results – they reflect a different pipeline (cc1.py) that computed rolling/lag features before the train/test split, introducing mild look-ahead leakage. Under the current leakage-safe protocol, the honest RMSE values for the same algorithms are 5,348 and 5,387 respectively. The canonical Review II table uses only the leakage-safe fresh rerun numbers.

8.5 Prophet

Prophet was successfully evaluated in the Review II run, achieving RMSE 37,717, MAE 36,769, MAPE 64.2%, and R^2 −38.99 – substantially worse than all other models including the statistical baselines. This outcome is consistent with the literature: Prophet's additive decomposition model (trend + seasonality + holidays) assumes relatively smooth, consistent trend components. The LCL fleet-level daily series exhibits high day-to-day variability (IQR spanning tens of thousands of kWh) that overwhelms the smooth component model. Additionally, Prophet is not given access to the engineered lag and rolling features available to the tree-based models, placing it at a structural disadvantage in this comparison.

Prophet results are included in the canonical comparison table.

8.6 Limitations

1. SARIMA and ETS were not evaluated in the canonical rerun; they represent directions for future extension of this comparison.

9. Conclusion

This project delivers a complete, rerun-backed academic package for Review II on the problem of daily energy consumption forecasting from London smart meter data.

9.1 Key Findings

1. **CatBoost is the best-performing model** in the canonical fresh rerun, achieving RMSE 5,151 kWh and MAE 1,907 kWh on the 112-day test window – a 66.8% improvement in RMSE over the Naive baseline.

2. **All gradient-boosted tree models substantially outperform baselines**, with LightGBM and XGBoost following CatBoost closely. The performance margins among the top three are small, indicating robust feature utilisation rather than model-specific overfitting.

3. **Deep learning models (GRU, BiLSTM) are competitive but trail tree-based models** on this relatively small dataset (517 training rows). With a larger dataset and GPU acceleration, the gap could narrow.

4. **Leakage-safe evaluation is critical**: the legacy pipeline's slightly lower RMSE values (≈4,500 kWh) resulted from computing rolling features before the train/test split, a subtle but impactful leakage pattern. The canonical pipeline eliminates this.

5. **Lag and rolling features dominate** all tree-based models, confirming strong short-range autocorrelation in daily energy consumption.

9.2 Future Work

1. Extend the model set with SARIMA, N-BEATS, or Transformer architectures under the same leakage-safe protocol.
2. Explore per-household probabilistic forecasting using quantile regression or normalising flows.
3. Investigate the effect of weather covariates (temperature, cloud cover) as external regressors.

9.3 Summary

The Review II package is complete: all code, data, model artefacts, evaluation tables, and visualisations are present in the repository under version control. The primary results are fresh-rerun backed, clearly labelled, and traceable to specific model artefacts in `models/final/`. The inherited legacy results are preserved separately in `results/inherited/` for provenance without contaminating the main comparison table.

References

[1] UK Power Networks, *Low Carbon London: A Learning Journey*, UK Power Networks, 2014.

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.

[3] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[4] A. V. Dorogush, V. Ershov, and A. Gulin, "CatBoost: Gradient Boosting with Categorical Features Support," *arXiv preprint arXiv:1810.11363*, 2018.

[5] S. J. Taylor and B. Letham, "Forecasting at Scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[6] K. Cho *et al.*, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," in *Proc. EMNLP*, 2014.

[7] M. Schuster and K. K. Paliwal, "Bidirectional Recurrent Neural Networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, 1997.

[8] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in NeurIPS*, 2019.

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.

Document generated automatically from verified experimental results.
All metrics are from fresh reruns on `results/final/`*. Device: CPU (torch 2.10.0+cpu).*
Best model: CatBoost (RMSE 5,151). All 9 models completed successfully.